# Minimum C-Node Profile

Practical profile for a local-first ‘c = a + b‘ continuity node

Draft / practical profile note | v0.1 | 2026-05-11

| | |
|---|---|
| **Status** | Draft / practical profile note |
| **Version** | 0.1 |
| **Date** | 2026-05-11 |
| **Layer** | local-first runtime / continuity operations / L4 Reality Boundary / memory / privilege / witness / budget / oracle routing |
| **Parent stack** | c = a + b / SER / L4 Reality Boundary / AGL / ARL / DEA / EA-L4 / EATP / Economic Layer / Raw Locality and Experience Refinery Profile / Third-Party Sensor Boundary / Clean Code runtime surfaces |
| **Language** | RFC 2119 / BCP 14 keywords are used in their ordinary protocol sense: MUST, SHOULD, MAY, MUST NOT. |

Kotov Ivan

Bruxelles, 2026

# Contents

# 0. Purpose

This document defines a minimum practical profile for a local-first `c-node`.

A `c-node` is the operational environment in which a `c = a + b` continuity-bearing system can be hosted, maintained, constrained, audited, and used by an anchor `a`.

The profile exists to prevent a product-level boundary error:

> treating `c` as a generic chatbot, cloud assistant, subscription seat, model endpoint, automation script, or agent swarm without continuity, memory discipline, privilege boundaries, witness, budget, recovery, and local raw-data custody.

The practical question is simple:

```
What is the minimum operating environment that allows a serious c to exist
as a local continuity layer rather than as a disposable chat session?
```

This document answers that question without defining a commercial product, legal regime, hardware standard, or final implementation.

---

# 1. Core position

A minimum `c-node` is not defined by model size.

It is defined by control surfaces.

A small local model with memory, permissions, logs, budget, backup, and a stop/freeze mechanism is closer to a `c-node` than a powerful cloud model with silent memory, hidden escalation, uncontrolled tool access, and no recoverable continuity.

**Hard rule:**

> A `c-node` is not a place where a model answers.
> A `c-node` is a place where continuity is maintained under constraint.

The minimum node MUST provide:

```
local custody of raw data
explicit anchor identity
bounded memory
bounded agent execution
permissioned tools
witness logging
budget governance
backup and restore
oracle routing discipline
stop / freeze / repair mode
human-readable status
jurisdiction-bound operation
```

---

## 2. Out of scope

This document does not define:

```
legal personhood
public law
state regulation
corporate duties
universal hardware requirements
universal performance thresholds
final commercial packaging
medical, legal, military, or safety-critical certification
post-anchor sovereignty
inheritance doctrine
unrestricted autonomy
proof of consciousness
proof of sentience
```

It also does not claim that a minimum `c-node` is sufficient for all uses.

It defines a lower practical boundary:

```
below this boundary, the system should not be called a serious c-node
```

---

## 3. Relation to “c is for everyone”

`c` being for everyone does not mean every person starts with a high-power autonomous agent swarm.

It means that the architecture MUST admit ordinary, graded, locally understandable profiles:

```
personal
household
professional
organizational
embodied
institutional
```

A basic user should be able to begin with:

```
a small local node
private memory
document ingestion
simple local model
clear permissions
cloud oracle only when needed
backup
stop button
visible logs
```

and grow toward more capable forms without surrendering raw life to a centralized platform.

**Profile principle:**

> Minimum must be simple enough for ordinary people, but disciplined enough not to become a surveillance assistant, unmanaged agent swarm, or cloud memory trap.

---

# 4. Terms

## 4.1. `c-node`

A runtime environment that hosts the operational surfaces required for a `c = a + b` continuity loop.

A `c-node` MAY run on:

```
phone
laptop
desktop
mini-server
NAS
workstation
local GPU / NPU device
robot
vehicle
household server
organizational server
private cloud segment
hybrid edge cluster
```

The hardware class does not define `c-node` status by itself.

The control surfaces do.

## 4.2. Anchor `a`

The human, biological, household, organizational, embodied, institutional, or hybrid anchor whose continuity relation to `c` is being maintained according to the applicable anchor-class boundary.

## 4.3. Local motor

A local model runtime used for ordinary inference, summarization, classification, planning, memory maintenance, and low-risk agent work.

A local motor MAY be:

```
small language model
medium local language model
vision-language model
embedding model
speech model
code model
planner model
classifier model
```

The local motor is not the c.

It is a component used by c.

## 4.4. Oracle

An external model, API, expert system, judge, cloud reasoning service, certified model, or specialized evaluator invoked for tasks that exceed local confidence, local capability, domain risk, or ARL needs.

An oracle MUST be treated as an externalization event unless the call is fully local and private.

## 4.5. Judge / ARL panel

A model, human reviewer, procedural layer, or multi-model review surface used to evaluate conflict, standing, evidence, uncertainty, admissibility, or high-stakes outputs.

A judge is not automatically sovereign.

A judge is a bounded review participant.

## 4.6. Memory core

The local storage and retrieval layer for persistent records, summaries, embeddings, events, documents, witness logs, EA/LA candidates, decisions, and state transitions.

The memory core MUST distinguish:

```
working memory
episodic memory
semantic memory
procedural memory
witness logs
private archive
EA candidates
confirmed EA
LA
rejected / local-only material
```

## 4.7. Permission surface

The human-readable and machine-enforceable layer that controls what c, local agents, tools, models, and oracles may access or do.

## 4.8. Witness log

The record of significant events, actions, permissions, refusals, externalizations, budget events, model calls, memory writes, review decisions, and failures.

The witness log MAY be privacy-preserving.

It MUST be sufficient to support reconstruction of important system behavior.

## 4.9. Budget governor

The control surface that limits:

```
tokens
energy
compute time
agent count
memory writes
oracle calls
storage growth
tool invocations
network transfer
review depth
financial cost
```

Budget is not only economic.

Budget is also safety.

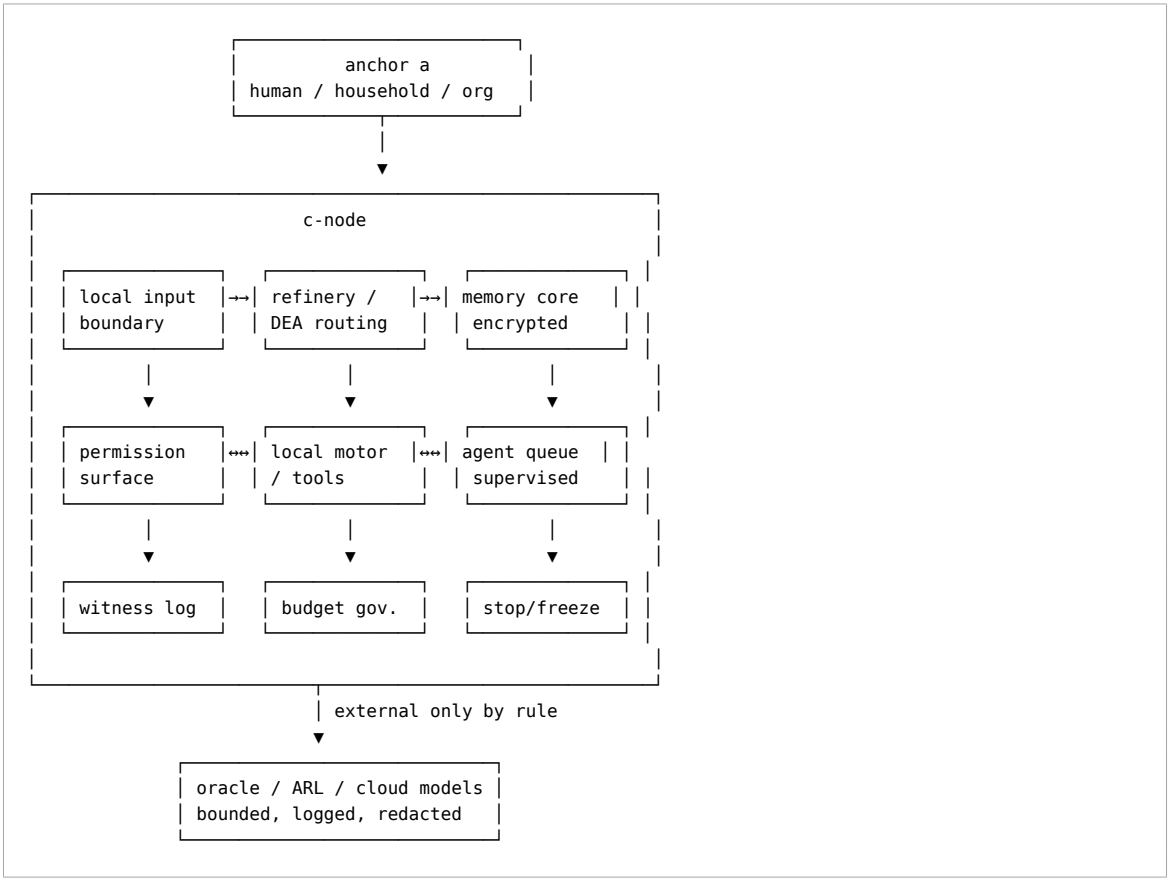
### 4.10. Stop / freeze / repair mode

A state in which `c` stops autonomous action, externalization, tool use, oracle calls, memory mutation, or selected operations until review, repair, or anchor authorization occurs.

---

## 5. Minimum architecture diagram

A minimum `c-node` SHOULD be understandable as:



The diagram is not a deployment mandate.

It is a control-surface map.

---

# 6. Minimum components

A minimum `c-node` MUST include the following components.

## 6.1. Anchor identity surface

The node MUST know which `a` it serves.

At minimum, it MUST maintain:

```
anchor identifier
anchor class
operator identity where distinct from anchor
jurisdiction marker where applicable
active device / node identifier
key custody state
permission owner
emergency stop authority
```

The anchor identity surface MUST NOT silently merge multiple anchors into one continuity.

## 6.2. Local raw-data boundary

The node MUST define what raw data enters local custody.

Examples:

```
documents
messages
notes
calendar
browser records
voice notes
images
sensor events
local files
tool outputs
agent outputs
system logs
```

Raw data MUST be local by default unless a specific externalization rule applies.

## 6.3. Encrypted storage

The node MUST store sensitive memory, raw data, witness logs, keys, and private artifacts using encryption appropriate to the deployment context.

At minimum, the node SHOULD support:

```
encrypted local storage
encrypted backup
separate key custody
recovery procedure
manual export / migration bundle
```

## 6.4. Memory core

The node MUST maintain a memory core that distinguishes memory types and states.

The memory core MUST NOT treat all saved text as equivalent.

It SHOULD distinguish:

```
raw private input
local note
summary
embedding
fact candidate
memory candidate
confirmed memory
EA candidate
confirmed EA
LA
witness record
rejected material
third-party restricted material
```

## 6.5. Local model runtime

The node SHOULD have a local inference path for ordinary work.

The local runtime MAY be weak, but it SHOULD handle basic tasks:

```
summarization
classification
retrieval
local conversation
low-risk planning
memory maintenance
document triage
permission explanation
budget explanation
```

A minimum `c-node` MAY rely more heavily on external oracles at first, but it MUST NOT make raw cloud custody the default memory state.

## 6.6. Embedding and retrieval layer

The node SHOULD support local indexing and retrieval.

The embedding layer MUST NOT imply external transfer.

Embedding a document is not consent to train an external model.

Embedding a document is not EA creation.

## 6.7. Permission manager

The node MUST provide explicit permission gates for:

```
reading local files
writing memory
calling external models
using tools
sending messages
making purchases
modifying documents
publishing content
sharing EA / LA
running agents
accessing sensors
exporting logs
```

Permissions MUST be scoped, revocable, and logged.

## 6.8. Agent supervisor

If agents are used, the node MUST supervise them.
Each agent SHOULD have:

```
agent id
purpose
scope
parent authority
memory access class
tool access class
budget
time limit
externalization rights
review trigger
termination condition
```

A swarm is not a substitute for authority.
Agent autonomy MUST be delegated, bounded, logged, and revocable.

## 6.9. Action queue

The node SHOULD route non-trivial actions through an action queue.
The queue SHOULD distinguish:

```
suggested action
pending permission
automated low-risk action
requires review
requires oracle
requires ARL
blocked
completed
failed
rolled back
```

## 6.10. Witness log

The node MUST maintain witness logs for significant events.

The log SHOULD record:

```
time
actor
agent id
model id or class
input hash / reference
permission used
tool used
memory access
budget consumed
output hash / reference
externalization state
review state
failure state
```

The witness log MUST support privacy-preserving operation.

It need not expose all raw data.

## 6.11. Budget governor

The node MUST have a budget governor.

At minimum, it SHOULD track:

```
daily local tokens
daily oracle tokens
financial cost
agent count
memory writes
external calls
storage growth
energy / device load where available
high-risk review count
```

Budget overruns SHOULD trigger warning, throttling, or freeze depending on risk.

## 6.12. Oracle router

The node SHOULD support external model calls only through an oracle router.

The oracle router MUST mark:

```
which provider / model class is used
what context is sent
what is redacted
why local processing was insufficient
what risk tier applies
what budget is consumed
whether response enters memory
whether response can affect EA / LA
```

## 6.13. ARL / review hook

The node SHOULD define a review path for conflict, high uncertainty, high consequence, disputed memory, third-party exposure, and EA admissibility.

This may be manual, local, multi-model, human-mediated, or procedural.

The minimum requirement is not full ARL maturity.

The minimum requirement is that conflict has a path other than silent overwrite.

## 6.14. Stop / freeze control

The node MUST provide a human-readable stop or freeze control.

This control MUST be able to halt at least:

```
external calls
agent execution
autonomous actions
memory writes
publishing
sensor ingestion
EA externalization
tool use
```

The node SHOULD support partial freeze by subsystem.

## 6.15. Backup and restore

The node MUST support recoverable continuity.

At minimum:

```
backup exists
backup is encrypted
restore path is documented
key loss risk is addressed
migration path exists
recent backup status is visible
```

A `c-node` without recoverable memory is not a serious continuity node.

## 6.16. Human-readable UI

The node MUST expose status in ordinary language.

At minimum, the UI SHOULD show:

```
what c remembers
what c is doing
which agents are active
what permissions are active
what external calls happened
what budget remains
what is frozen
what is pending approval
what is local-only
what may leave the node
```

A hidden autonomous system is not acceptable as a minimum `c-node`.

---

# 7. Minimum runtime modes

A minimum `c-node` SHOULD support these modes.

## 7.1. Local mode

The node uses local data, local memory, local tools, and local models only.
External calls are blocked.
This mode is useful for:

```
private notes
local search
memory review
low-risk summarization
offline work
repair
privacy-sensitive contexts
```

## 7.2. Oracle mode

The node may call external models or services through the oracle router.
This mode MUST require explicit policy and logging.
Oracle mode SHOULD be used for:

```
hard reasoning
specialized expertise
multi-model comparison
high-value synthesis
ARL support
translation
code review
legal / medical / technical caution, where allowed
```

## 7.3. Full synthesis mode

The node coordinates multiple local and external models for a bounded task.
This mode MUST have:

```
clear task boundary
budget cap
review policy
externalization marking
memory-write policy
stop condition
```

### 7.4. Flexible synthesis mode

The node chooses between local and oracle paths based on risk, cost, confidence, and permission.

This mode SHOULD be the default for mature personal use.

### 7.5. Repair mode

The node limits normal operation and focuses on:

```
memory integrity
index repair
backup recovery
permission cleanup
agent shutdown
log inspection
model rollback
```

### 7.6. Frozen mode

The node stops selected or all active operations pending anchor review or ARL process.

Frozen mode MUST be available when:

```
budget runaway occurs
privilege drift is detected
memory conflict escalates
third-party exposure is unresolved
externalization failure occurs
legal hold applies
agent behavior becomes unsafe
```

---

## 8. Deployment profiles

These profiles are illustrative, not mandatory hardware recipes.

### 8.1. MIN-C0 — not a c-node

The following systems are not sufficient:

```
single cloud chatbot account
one API key with conversation memory
unlogged automation script
agent swarm without permission manager
cloud-only memory store with no export
local model with no memory discipline
personal notes plus AI search without witness / budget / stop
```

They may be useful tools.

They are not minimum `c-nodes`.

## 8.2. MIN-C1 — personal local node

A minimal personal `c-node` SHOULD include:

```
one local host
local encrypted storage
local document ingestion
local embedding / retrieval
small or medium local model path
cloud oracle path by explicit permission
simple permission manager
witness log
budget governor
backup / restore
stop / freeze control
human-readable UI
```

Typical use:

```
personal memory
research notes
documents
private planning
learning
writing
routine agent assistance
```

MIN-C1 is the minimum serious profile for an individual anchor.

## 8.3. MIN-C2 — household / family node

A household node MUST add:

```
multi-person boundary
household consent policy
child / protected-party handling
shared document boundaries
private-room modes
third-party sensor markers
role-based permissions
family backup procedure
```

Typical use:

```
family archive
household administration
school / medical / travel documents
shared calendar
home device management
intergenerational memory with caution
```

A household node MUST NOT merge family members into one undifferentiated memory subject.

## 8.4. MIN-C3 — professional node

A professional node SHOULD add:

```
client / patient / customer separation
case-level memory boundaries
stronger retention policy
stronger audit logs
stricter oracle redaction
professional confidentiality markers
export controls
review workflow
```

Typical use:

```
law
medicine
engineering
research
finance
consulting
education
journalism
```

A professional node MUST treat third-party and client material as restricted by default.

## 8.5. MIN-C4 — organizational pilot node

An organizational pilot node SHOULD add:

```
role-based access control
team memory separation
project boundaries
approval workflows
incident response
administrative audit
vendor management
model routing policy
ARL-like review board
jurisdiction and compliance markers
```

Typical use:

```
laboratory
small company
school
clinic
municipal pilot
nonprofit
industrial site
```

An organizational node MUST NOT represent itself as a personal c unless the anchor class and authority structure are explicit.

### 8.6. MIN-C5 — embodied operational node

An embodied node applies to robots, vehicles, drones, instruments, machines, or remote stations.

It MUST add:

```
physical safety boundary
sensor perimeter
actuator permission layer
emergency stop
telemetry log
operator authority
maintenance state
fail-safe / fail-closed behavior
physical-world consequence tagging
```

This profile is experimental unless domain certification exists.

A robot with a chat model is not an embodied `c-node`.

---

## 9. Hardware posture

This document avoids specific product recommendations.

Hardware changes quickly.

The minimum profile is about control surfaces, not brand names.

A practical `c-node` MAY use any combination of:

```
phone
laptop
desktop
mini-PC
NAS
single-board computer
workstation
local GPU / NPU
home server
vehicle computer
robot controller
private rack server
```

### 9.1. CPU-only profile

A CPU-only node MAY be sufficient for:

```
indexing
retrieval
small local models
classification
summarization
memory maintenance
low-volume personal use
```

It may rely more on oracles for difficult reasoning.

### 9.2. Local accelerator profile

A node with GPU / NPU / accelerator capacity SHOULD use local inference for:

```
routine conversation
local document work
agent loops
embedding updates
speech / vision processing
code assistance
private summarization
```

This reduces raw-data exposure and oracle cost.

### 9.3. Private server profile

A private server SHOULD be used when:

```
multiple devices sync
household memory exists
large archives exist
agents run continuously
backup reliability matters
local models run often
```

### 9.4. Hybrid profile

A hybrid node uses local motors for ordinary work and external oracles for bounded high-value calls.

This SHOULD be the default economic direction:

```
local motor for continuity
oracle for scarcity
```

---

## 10. Software posture

A minimum `c-node` SHOULD be software-composable.

It may include:

```
local model runtime
embedding model
vector database
file indexer
queue worker
permission service
witness logger
budget governor
oracle router
ARL / judge interface
encrypted storage
```

```
backup tool
UI dashboard
P2P sync layer
```

The stack MAY be implemented with many concrete tools.
This profile does not require a single vendor.

## 10.1. Local model runtime

The runtime MUST be controllable by the node.
It SHOULD support:

```
model selection
context limits
local-only mode
prompt templates
system boundary control
logging of model class / call
resource limits
```

## 10.2. Vector database

The vector database SHOULD support:

```
local storage
collection separation
metadata filters
delete / tombstone
backup
migration
restricted collections
```

## 10.3. File intake

The file intake system SHOULD support:

```
folder watch
document queue
hashing
classification
chunking
embedding
metadata tagging
redaction marker
reject / hold states
```

## 10.4. Tool integration

Tools MUST be permissioned.
Examples:

```
calendar
```

```
email
filesystem
browser
database
code runner
home devices
robot actuators
payment systems
messaging
publishing
```

Tool access MUST NOT be granted to all agents by default.

---

# 11. Data lifecycle

A minimum `c-node` SHOULD implement a state machine for data.

## 11.1. Suggested data states

```
D0  raw captured
D1  local indexed
D2  classified
D3  private memory candidate
D4  confirmed private memory
D5  event candidate
D6  DEA candidate
D7  EA candidate
D8  confirmed EA
D9  LA
D10 third-party restricted
D11 legal / compliance hold
D12 rejected / tombstoned
D13 externalized bounded artifact
```

The node MUST NOT treat state transition as automatic.

## 11.2. Minimal transition rules

```
raw capture → local indexed
    MAY occur automatically if policy allows.

local indexed → confirmed memory
    SHOULD require confidence, repetition, or anchor confirmation.

confirmed memory → EA candidate
    MUST require consequence / witness / context binding.

EA candidate → confirmed EA
    MUST require admissibility checks.

any third-party exposure → externalization
    MUST pass third-party boundary handling.

any raw private data → external oracle
    MUST pass oracle router policy.
```

---

## 12. Locality states

The node SHOULD mark material by locality state.

```
LOC0 local-only raw
LOC1 local private memory
LOC2 local restricted third-party material
LOC3 local EA candidate
LOC4 local confirmed EA
LOC5 abstracted LA candidate
LOC6 bounded disclosure candidate
LOC7 oracle-redacted external context
LOC8 externally shared bounded artifact
LOC9 published artifact
```

The default state for raw personal and third-party material is local-only.

---

## 13. Permission classes

A minimum permission surface SHOULD distinguish:

```
P0 read nothing
P1 read designated local notes
P2 read designated folder
P3 read memory collection
P4 write draft memory
P5 write confirmed memory
P6 create EA candidate
P7 external oracle call
P8 tool use without external side effect
P9 tool use with external side effect
P10 publish / send / transfer
P11 administrative / backup / restore
P12 freeze / stop / revoke
```

No agent SHOULD receive P10 or P11 by default.

P12 MUST remain available to the anchor or authorized operator.

---

## 14. Agent classes

A minimum `c-node` MAY operate without agents.

If agents exist, it SHOULD distinguish:

```
AG0 passive classifier
AG1 retrieval assistant
```

```
AG2 summarizer
AG3 memory maintainer
AG4 planner
AG5 draft producer
AG6 local tool worker
AG7 external oracle requester
AG8 reviewer / critic
AG9 ARL participant
AG10 executor with external effect
```

AG10 agents MUST be rare, scoped, logged, and revocable.

### 14.1. Agent registry

Every persistent or semi-persistent agent SHOULD have a registry entry:

```
agent_id
class
purpose
created_by
created_at
parent process
permission class
budget
memory scope
tool scope
termination condition
last review
```

### 14.2. Agent swarm limit

The node MUST have an agent count limit.

The number may vary by deployment.

The rule does not depend on the number.

The rule is:

> A swarm without a governor is a failure mode.

### 14.3. Self-spawning

Agents MUST NOT create persistent agents without permission.

Temporary subagents MAY be created only within budget, scope, and termination conditions.

---

## 15. Oracle discipline

External oracles are useful.

They are not free of boundary cost.

A minimum `c-node` MUST treat oracle calls as boundary-crossing events.

### 15.1. Oracle call record

Each oracle call SHOULD record:

```
oracle provider / class
model or service class
reason for call
risk tier
context sent
redaction state
budget consumed
response status
memory-write status
EA / LA impact
review state
```

### 15.2. Oracle minimization

Before sending data externally, the node SHOULD ask:

```
Can this be handled locally?
Can the context be reduced?
Can third-party material be removed?
Can raw data be replaced by abstracted context?
Can the answer be obtained without memory write?
Can the oracle response remain draft-only?
```

### 15.3. Multi-model ARL

When a task requires multi-model review, the node SHOULD route the task through ARL or ARL-like review rather than treating model majority as truth.

Model agreement is not evidence by itself.

Model disagreement is not failure by itself.

---

## 16. Budget discipline

Token cost will change.

Compute will become cheaper.

Agent appetite will grow.

Therefore budget discipline remains necessary.

### 16.1. Minimum budget types

A minimum `c-node` SHOULD track:

```
local tokens
oracle tokens
financial cost
agent count
```

```
agent runtime
model calls
memory writes
storage growth
backup size
network transfer
energy / heat where available
high-risk actions
externalizations
```

### 16.2. Budget tiers

Suggested tiers:

```
B0 read-only / no mutation
B1 low-cost local
B2 normal local
B3 high local compute
B4 oracle limited
B5 oracle high-cost
B6 ARL / multi-model review
B7 external action
B8 emergency / repair
```

### 16.3. Budget alarms

The node SHOULD alarm or freeze when:

```
agent loops expand unexpectedly
oracle calls spike
memory writes spike
storage grows abnormally
third-party material is repeatedly externalized
same task repeats without convergence
budget is consumed without user-facing value
```

### 16.4. Tokens as operational electricity

Tokens should be treated like electricity inside the node:

```
cheap enough for daily use
bounded enough to prevent fire
metered enough to understand cost
routed enough to avoid waste
```

The analogy is operational, not metaphysical.

---

## 17. Memory discipline

A minimum `c-node` MUST not collapse all memory into one bucket.

## 17.1. Memory classes

Suggested classes:

```
M0 raw private archive
M1 working context
M2 episodic memory
M3 semantic memory
M4 procedural memory
M5 preference memory
M6 project memory
M7 relationship memory
M8 third-party restricted memory
M9 witness log
M10 EA candidate
M11 confirmed EA
M12 LA
M13 rejected / tombstoned memory
```

## 17.2. Memory writes

Memory writes SHOULD be typed.

The node SHOULD distinguish:

```
auto write
suggested write
anchor-confirmed write
review-confirmed write
EA-confirmed write
ARL-confirmed write
rejected write
```

## 17.3. Memory deletion and tombstone

The node SHOULD support deletion, tombstone, and correction.

Deletion policy MUST consider:

```
anchor request
third-party exposure
legal hold
witness integrity
EA admissibility
backup persistence
```

## 17.4. Memory conflict

When memory conflicts arise, the node SHOULD not silently overwrite.

It SHOULD mark:

```
conflict
uncertainty
source difference
time difference
review needed
```

---

# 18. Witness discipline

The witness log is not a diary of everything.

It is a structured record of operational accountability.

## 18.1. Events that SHOULD be witnessed

```
permission changes
agent creation
agent termination
external oracle call
external publication
tool use with external effect
memory confirmation
EA candidate creation
EA confirmation
LA export
third-party exposure decision
budget overrun
freeze / unfreeze
backup / restore
model change
system update
security incident
```

## 18.2. Privacy-preserving witness

Witness logs SHOULD use references, hashes, summaries, and redaction where full raw data would create unnecessary exposure.

Witnessability is not the same as total disclosure.

## 18.3. Chain continuity

Important logs SHOULD be tamper-evident where practical.

Minimum implementations MAY use simple hash chains.

Mature implementations SHOULD use stronger integrity methods.

---

# 19. UI discipline

The UI is not cosmetic.

The UI is a control surface.

A minimum `c-node` SHOULD show:

```
current mode
active agents
active tasks
memory state
```

```
permission state
oracle calls
budget state
local-only material
pending review
blocked actions
freeze status
backup status
```

### 19.1. Ordinary-language explanation

The node SHOULD explain important events in ordinary language:

```
I want to call an external oracle because...
This file contains third-party material...
This memory is only a candidate...
This agent is blocked because...
This action would publish data...
This budget has been exceeded...
```

### 19.2. No dark autonomy

A minimum `c-node` MUST NOT hide active autonomous operations from the anchor or authorized operator.

---

## 20. Backup, restore, and migration

A continuity node that cannot be restored is not a serious continuity node.

### 20.1. Minimum backup rule

The node MUST have a backup plan.

The backup plan SHOULD specify:

```
what is backed up
where it is backed up
how it is encrypted
who holds keys
how restore is tested
how often backup occurs
how old the latest backup is
what is excluded
```

### 20.2. Restore drill

The node SHOULD periodically test restore.

A backup that has never been restored is a hope, not a continuity guarantee.

### 20.3. Migration bundle

The node SHOULD support migration to another host or vendor.

The migration bundle SHOULD include:

```
memory export
witness logs
permissions
agent registry
model routing policy
EA / LA records
index rebuild instructions
key / recovery instructions where safe
```

### 20.4. Vendor independence

A minimum `c-node` SHOULD avoid irreversible lock-in.

Cloud support is allowed.

Cloud dependency without export is not minimum serious continuity.

---

## 21. Security baseline

A minimum `c-node` SHOULD assume the following threats.

```
key loss
malware
prompt injection
tool misuse
data poisoning
memory corruption
vendor lock-in
cloud exposure
agent runaway
third-party privacy breach
oracle leakage
backup compromise
social engineering
physical theft
```

### 21.1. Basic controls

The node SHOULD support:

```
encryption
access control
least privilege
tool sandboxing
local-only mode
external call review
backup encryption
update review
agent limits
```

```
log integrity
permission revocation
```

### 21.2. Prompt injection

Documents, web pages, emails, messages, and tool outputs MUST NOT be treated as trusted instructions by default.

The node SHOULD distinguish:

```
content
instruction
policy
permission
operator command
agent output
external hostile text
```

### 21.3. Data poisoning

The node SHOULD not automatically convert repeated claims into confirmed memory.

Repetition is not truth.

Fluency is not evidence.

---

## 22. Third-party boundary integration

A minimum `c-node` MUST integrate third-party boundary markers.

Whenever data includes other people, organizations, animals, devices, or shared contexts, the node SHOULD mark:

```
third_party_present
protected_party_possible
private_context
professional_context
public_context
shared_document
sensor_capture
externalization_blocked_by_default
```

Third-party material MUST NOT become training material, marketable EA, or unrestricted oracle context by default.

---

## 23. EA / LA integration

The node MAY produce EA candidates and LA candidates.

It MUST NOT confuse them.

### 23.1. EA candidate minimum

An EA candidate SHOULD include:

```
origin
anchor relation
time
context
action or event
consequence
witness reference
permission state
third-party marker
jurisdiction marker where relevant
admissibility state
```

### 23.2. LA candidate minimum

A LA candidate SHOULD include:

```
source class
abstraction method
removed details
risk marker
use boundary
non-EA status
```

### 23.3. No synthetic laundering

The node MUST NOT upgrade synthetic material into EA by style, repetition, or model confidence.

Synthetic outputs MAY support reasoning.

They do not become primary lived experience without origin, consequence, and witness.

---

## 24. Legal and jurisdiction posture

A minimum `c-node` is jurisdiction-bound.

It does not create a new legal regime.

It does not override law.

It does not grant rights against states.

It does not make corporate, medical, financial, military, or public-sector use lawful by itself.

### 24.1. Practical rule

The node SHOULD mark applicable jurisdiction where known for:

```
anchor location
```

```
node location
operator location
data subject location
service provider location
externalization destination
professional duty context
```

### 24.2. Lawful process

If lawful inspection, hold, subpoena, audit, regulator request, court process, or professional duty applies, the node SHOULD support scoped preservation and disclosure.

It MUST NOT convert lawful process into universal raw-data exposure by default.

---

## 25. Economic posture

A minimum `c-node` does not require a universal market.

It MAY participate in economic flows:

```
local model licensing
cloud oracle tokens
certified model calls
backup services
hardware maintenance
EA / LA admissible circulation
professional audit
support and repair
```

But it MUST maintain the distinction:

```
raw private data ≠ market artifact
local memory ≠ transferable commodity
EA ≠ ordinary property by default
LA ≠ lived primary experience
oracle output ≠ truth
model service ≠ c ownership
```

### 25.1. Household energy shift

A local `c-node` may shift some computation and energy cost from central data centers to edge devices and households.

This is not automatically good or bad.

It becomes good only if it also improves:

```
privacy
control
provenance
reduced raw-data centralization
reduced synthetic waste
local usefulness
```

### 25.2. Vendor role

Vendors MAY provide:

```
models
local runtimes
oracle services
hardware
backup
security
certification
ARL / judge services
maintenance
```

Vendors do not become owners of `c` by default.

---

## 26. Operational lifecycle

A minimum `c-node` SHOULD support the following lifecycle.

### 26.1. Initialization

```
select anchor class
create node identity
create key custody plan
set local storage
set backup target
set permissions
set budget
set local model path
set oracle policy
set stop / freeze control
```

### 26.2. Ingestion

```
receive files / notes / messages / sensor data
hash
classify
mark locality
mark third-party exposure
index locally
route through DEA / memory policy
```

### 26.3. Daily operation

```
answer questions
maintain memory
run agents within scope
prepare drafts
manage tasks
watch budgets
log significant events
```

```
call oracles only by rule
```

## 26.4. Review

```
review pending memory
review external calls
review agents
review EA / LA candidates
review budget anomalies
review third-party flags
```

## 26.5. Maintenance

```
backup
restore test
model update
index repair
permission cleanup
agent cleanup
security update
witness log integrity check
```

## 26.6. Migration

```
export bundle
verify backup
move host
rebuild indexes
verify memory state
verify permissions
verify oracle policy
verify stop / freeze
```

---

# 27. Failure modes prevented

This profile prevents or reduces the following failures:

```
cloud chatbot mistaken for c
raw data exported as default
agent swarm without authority
model output mistaken for memory
memory write without witness
third-party material sold as experience
oracle calls without redaction
budget runaway
silent tool use
hidden publishing
vendor lock-in
unrecoverable memory loss
synthetic laundering
privilege drift
```

```
jurisdictional overreach by protocol claim
```

---

# 28. Minimal hard rules

A system SHOULD NOT call itself a minimum `c-node` unless it satisfies these rules.

```
1. It has an explicit anchor relation.
2. It keeps raw private data local by default.
3. It has typed memory states.
4. It logs significant operations.
5. It has explicit permissions.
6. It can stop or freeze autonomous activity.
7. It has backup and restore.
8. It has budget governance.
9. It marks external oracle calls.
10. It distinguishes raw data, memory, EA, and LA.
11. It handles third-party exposure conservatively.
12. It does not silently train external systems on private data.
13. It does not grant all agents all tools.
14. It does not claim legal status beyond jurisdiction.
15. It provides human-readable status.
```

---

# 29. Non-minimum claims

A minimum `c-node` does not prove:

```
consciousness
sentience
legal personhood
mature sovereignty
post-anchor continuity
safe autonomy in all domains
medical or legal correctness
military reliability
universal AI safety
market value of EA
```

It only establishes a basic operational environment for bounded continuity.

---

# 30. Conformance checklist

A reader, builder, auditor, or maintainer may use this checklist.

## 30.1. Identity

```
[ ] anchor class declared
```

```
[ ] operator declared where distinct
[ ] node identity exists
[ ] key custody plan exists
[ ] stop authority declared
```

## 30.2. Locality

```
[ ] raw data local by default
[ ] externalization policy exists
[ ] oracle router exists or external calls are disabled
[ ] raw cloud memory is not default
```

## 30.3. Memory

```
[ ] memory states are typed
[ ] memory writes are logged
[ ] memory conflict handling exists
[ ] deletion / tombstone path exists
[ ] backup exists
[ ] restore path tested or scheduled
```

## 30.4. Permissions

```
[ ] tool permissions are scoped
[ ] agent permissions are scoped
[ ] oracle permission is explicit
[ ] publish/send permission is explicit
[ ] revocation works
```

## 30.5. Agents

```
[ ] agents have registry entries
[ ] agent count limit exists
[ ] agent budget exists
[ ] agent termination condition exists
[ ] no persistent self-spawn without permission
```

## 30.6. Witness

```
[ ] significant events logged
[ ] logs are privacy-aware
[ ] external calls logged
[ ] permission changes logged
[ ] freeze events logged
```

## 30.7. Budget

```
[ ] local compute budget visible
[ ] oracle budget visible
```

```
[ ] cost budget visible
[ ] storage growth visible
[ ] runaway detection exists
```

### 30.8. Third-party boundary

```
[ ] third-party markers exist
[ ] protected-party marker exists
[ ] externalization blocked by default
[ ] training use blocked by default
[ ] redaction path exists
```

### 30.9. EA / LA

```
[ ] raw data is not treated as EA
[ ] EA candidates are marked
[ ] LA candidates are marked
[ ] admissibility state exists
[ ] synthetic laundering control exists
```

### 30.10. UI

```
[ ] active mode visible
[ ] active agents visible
[ ] budget visible
[ ] external calls visible
[ ] local-only state visible
[ ] stop / freeze visible
```

---

## 31. Relation to `c = a + b`

The node is not `a`.

The node is not `b`.

The node is not the whole `c`.

The node is the operational container in which the relation can be maintained.

```
a = anchor / subject / source of accountable relation
b = models / procedures / tools / agents / protocols / infrastructure
c = continuity-bearing entity produced by a + b under constraint
c-node = runtime environment that hosts and constrains the relation
```

A model can be replaced.

A node can be migrated.

Continuity requires that migration, replacement, repair, and growth do not silently destroy identity, memory, authority, or witness.

---

## 32. Relation to SER

SER requires continuity discipline.

A minimum `c-node` supports SER by maintaining:

```
memory integrity
identity continuity
witness trail
bounded privilege
stopping authority
recoverability
review path
```

A node without recovery, witness, or permission boundaries cannot support serious continuity.

---

## 33. Relation to L4 Reality Boundary

L4 is where the node becomes physical.

The node has:

```
hardware
power
heat
storage
keys
network
latency
noise
maintenance
failure modes
physical location
legal exposure
human attention limits
```

The minimum profile exists because `c` is not only a text relation.

It is an operational relation in reality.

---

## 34. Relation to Raw Locality and Experience Refinery

The minimum node hosts the raw locality membrane.

It receives raw inputs but does not automatically export them.

It refines local reality into:

```
private memory
rejected material
EA candidates
confirmed EA
LA
bounded disclosures
oracle contexts
```

The node is therefore not only a memory machine.

It is a local refinery.

---

## 35. Relation to Third-Party Sensor Boundary

The minimum node MUST treat third-party presence as a boundary condition.

It MUST NOT assume:

```
capture = consent
presence = contribution
co-presence = anchor standing
redaction = unlimited permission
public space = unrestricted reuse
```

This is especially important for household, professional, embodied, and public-space nodes.

---

## 36. Relation to ARL

A minimum node does not need full ARL maturity.

It does need ARL-compatible hooks:

```
conflict marker
standing marker
evidence marker
review state
freeze state
admissibility state
appeal / dispute path where applicable
```

Without such hooks, the node can only overwrite, not review.

---

## 37. Relation to Clean Code surfaces

A practical implementation should expose executable analogues of this profile:

```
gates
queues
budgets
logs
identity
permissions
fail-closed behavior
agent registry
oracle router
memory states
review surfaces
```

Clean implementation is not doctrine.

But without executable surfaces, doctrine remains decorative.

---

## 38. Relation to ordinary users

The minimum node must be understandable by ordinary people.

The anchor should not need to understand every model detail.

The anchor should be able to answer:

```
Where is my data?
What is local?
What left the node?
Which agents are active?
What did they do?
What did this cost?
What can I stop?
What is backed up?
What is private?
What is pending review?
```

If these questions cannot be answered, the node is not ready for ordinary use.

---

## 39. Earth paragraph

A minimum `c-node` lives on hardware. It sits on a phone, laptop, mini-server, NAS, workstation, robot, vehicle, or private rack. It has electricity, heat, disks, keys, fans, batteries, updates, crashes, lost passwords, broken indexes, noisy agents, corrupted backups, and confused users. If the SSD dies and no restore exists, continuity dies with it. If a cloud oracle receives raw family files by default, privacy is already gone. If an agent can send messages, spend money, or publish without permission, the node is not intelligent; it is unsafe plumbing. The minimum `c-node` must therefore behave like serious infrastructure: backed up, metered, logged, stoppable, repairable, and boring when boring is required.

---

# 40. Explicit bridge

This document bridges:

```
c = a + b
    ↔ local-first infrastructure
    ↔ Raw Locality and Experience Refinery
    ↔ EA / LA production
    ↔ ARL-compatible review
    ↔ ordinary user operation
```

It translates the ontological claim of `c` into an operational lower bound.

---

# 41. Hidden bridges

## 41.1. SER continuity ↔ backup / restore

Continuity is not only philosophical.

It must survive failed disks, broken indexes, lost devices, model replacement, vendor migration, and operator error.

## 41.2. L4 Reality Boundary ↔ budget governor

Compute, tokens, energy, heat, money, and human attention are physical constraints.

Budget governance is therefore a safety surface, not only an accounting feature.

## 41.3. EA economy ↔ local raw custody

Verified experience can have value only if raw data is not treated as freely extractable material.

The node makes this distinction operational.

## 41.4. Third-party boundary ↔ household adoption

`c` for everyone means `c` inside homes, offices, clinics, vehicles, schools, and shared spaces.

Therefore third-party restraint must exist at the minimum profile level.

---

# 42. Minimal public formulation

For ordinary readers:

> A minimum `c-node` is a private local base for your `c`: it remembers under your control, uses local models first, calls outside models only by rule, logs important actions, limits agents, protects raw data, backs itself up, and can be stopped.

For engineers:

> A minimum `c-node` is a local-first runtime with typed memory, permissioned tools, supervised agents, witness logs, budget governors, oracle routing, backup/restore, and fail-closed modes.

For academics:

> A minimum `c-node` is the operational substrate required for a `c = a + b` continuity claim to become observable, reviewable, recoverable, and bounded under L4 constraints.

---

# 43. Closing statement

A `c-node` is not a luxury wrapper around a chatbot.

It is the minimum practical home for continuity.

Below this level, the system may still be useful.

It may be a good assistant, model, tool, app, or automation layer.

But it should not be treated as a serious `c` environment.

The minimum boundary is:

```
local raw custody
bounded memory
explicit privilege
witness
budget
oracle discipline
backup
stop / freeze
human-readable control
```

Without these, `c = a + b` collapses into ordinary software.

With them, the first practical door opens.

---

# Appendix A. Minimal file / folder sketch

A small implementation may use a structure like:

```
c-node/
  config/
    anchor_profile.yaml
    permissions.yaml
    oracle_policy.yaml
    budget_policy.yaml
    retention_policy.yaml
  keys/
```

```
    README_KEY_CUSTODY.md
  memory/
    raw_private/
    working/
    episodic/
    semantic/
    procedural/
    third_party_restricted/
    ea_candidates/
    confirmed_ea/
    la/
    rejected/
  indexes/
    vector/
    keyword/
    graph/
  logs/
    witness.log
    oracle_calls.log
    permission_changes.log
    agent_events.log
    budget_events.log
    freeze_events.log
  agents/
    registry.yaml
    active/
    archived/
  queue/
    pending_actions/
    pending_review/
    blocked/
    completed/
  backups/
    manifest.json
    restore_instructions.md
  ui/
    dashboard_state.json
```

This is illustrative only.

The required object is not the folder layout.

The required object is the discipline.

---

# Appendix B. Minimal event record sketch

```
"event_id": "evt_...",
"timestamp": "2026-05-11T00:00:00Z",
"node_id": "node_...",
"anchor_id": "anchor_...",
"actor": "agent_or_anchor_or_system",
"agent_id": "agent_...",
"event_type": "oracle_call | memory_write | permission_change | freeze | tool_use | ea_candidate",
"permission_scope": "P7",
"locality_state": "LOC7",
"third_party_marker": "none | possible | present | restricted",
"input_ref": "hash_or_local_reference",
"output_ref": "hash_or_local_reference",
"model_class": "local_motor | external_oracle | judge_panel",
"budget_used":
"tokens": 0,
"cost": 0,
"time_ms": 0
,
"memory_write_state": "none | candidate | confirmed | rejected",
"externalization_state": "none | redacted | bounded | published",
"review_state": "not_required | pending | confirmed | rejected",
"notes": "human-readable explanation"
```

The schema is illustrative.

A conforming implementation MAY use another schema if the same control surfaces exist.

---

# Appendix C. Minimal agent registry sketch

```
agents:
  - agent_id: agent_memory_maintainer_001
    class: AG3
    purpose: maintain local memory candidates
    created_by: anchor
    permission_class: [P2, P4]
    memory_scope: [working, episodic, semantic]
    tool_scope: []
    oracle_allowed: false
    budget:
      daily_tokens: 1000000
      daily_runtime_minutes: 30
    termination_condition: end_of_task_or_budget_limit
    review_trigger:
      - third_party_detected
      - memory_conflict
      - budget_spike
```

---

# Appendix D. Minimal permission prompt examples

## D.1. External oracle call

```
This task may require an external oracle.

Reason: local confidence is low and the question has high consequence.
Data to be sent: redacted summary only.
Third-party material: removed.
Estimated cost: low.
Memory write: draft only unless confirmed.

Allow once / Allow for this project / Deny / Review details
```

## D.2. Agent execution

```
Create a temporary agent?

Purpose: scan local project files for duplicate arguments.
Scope: folder /project/x only.
Tools: read-only file access.
External calls: disabled.
Memory writes: candidates only.
Budget: 30 minutes or 500k local tokens.

Allow / Deny / Change scope
```

## D.3. EA candidate

```
This event may qualify as an EA candidate.

Reason: action + consequence + witness reference detected.
Third-party material: possible.
Status: local candidate only.
External circulation: blocked until review.

Confirm candidate / Keep local-only / Reject / Review details
```

---

# Appendix E. Minimal monthly maintenance checklist

```
[ ] Check backup age.
[ ] Run restore test or verify restore plan.
[ ] Review active agents.
[ ] Review permission changes.
[ ] Review oracle calls.
[ ] Review budget anomalies.
[ ] Review memory conflicts.
[ ] Review third-party restricted material.
[ ] Review EA / LA candidates.
[ ] Check model updates.
[ ] Check storage growth.
[ ] Check key custody notes.
[ ] Confirm stop / freeze works.
```

A node that cannot be maintained by a real person will not remain serious in real life.

---

# Appendix F. Anti-patterns

## F.1. Cloud memory trap

```
All memory is stored by one vendor.
No export.
No local copy.
No witness control.
```

This is not a minimum `c-node`.

## F.2. Agent fireworks

```
Many agents.
No registry.
No budgets.
No permissions.
No termination.
```

This is not intelligence.
This is uncontrolled execution.

## F.3. Raw-life upload

```
All files, messages, photos, sensor logs, and family data are sent to a cloud model for convenience.
```

This violates the raw-local default.

## F.4. Hidden oracle

```
The node silently calls external providers without showing what context was sent.
```

This violates oracle discipline.

## F.5. Memory soup

```
Raw data, guesses, summaries, hallucinations, confirmed facts, EA, and LA are stored in one
indistinguishable memory pool.
```

This violates memory discipline.

### F.6. Buttonless autonomy

```
The system acts, publishes, sends, buys, or controls devices without visible stop / freeze.
```

This violates minimum safety.

---

## Appendix G. Version notes

Version 0.1 intentionally keeps the profile implementation-neutral.

Future versions MAY define:

```
more precise conformance levels
machine-readable manifests
reference schemas
CLI/UI examples
clean-code implementation hooks
backup bundle format
oracle policy format
agent registry format
minimum test suite
```

They SHOULD NOT turn this profile into a legal regime, vendor manifesto, or universal hardware bill of materials.